Fines & Fees

Capstone project documentation

Frontend, user interface and FastAPI middleware

Prepared: 30 October 2026

1. Purpose and current readiness

The project centralizes customer fines and deductions across Logistics, Warehouse and Manufacturing. It replaces disconnected screens and browser-only mock services with a React interface connected to persistent FastAPI APIs.

The business problem is a manual investigation cycle reported to take 10-15 days. This implementation provides a searchable review queue, required decision comments, review history and financial visibility. It does not claim a measured reduction in turnaround time or automatically decide whether a fine is valid.

Your responsibility covers the React frontend, UI and FastAPI middleware. The database team will provide the validated dataset and SQL Server tables. The local implementation uses SQLite while that work is pending.

Architecture

```
React UI (browser)
    | HTTP requests with bearer session token
    v
FastAPI middleware
    | validation + role checks + business workflow
    v
SQLAlchemy data access
    | local today             | target integration
    v                         v
SQLite                    Microsoft SQL Server
                              ^
                              | managed using SSMS
```

React never connects directly to SQL Server. SSMS is the management application, not the database server. The API contract is the boundary between your frontend and the eventual database implementation.

Ready locally

• Three sign-in options with server-enforced role permissions.

• Signup pending administrator approval; approval and rejection APIs/UI.

• Filtered fines, details, summary cards, charts and source totals.

• Review decisions, required comments and version-protected audit history.

• Workbook validation, explicit transactional import and Excel export.

• USD reporting, responsive layouts and keyboard-aware dialogs.

Live SQL Server connection, final table mapping and real-data validation remain pending. This is a locally verified capstone implementation, not a production deployment certification.

2. Roles and application workflows

Page and action permissions

Page or action	User	Super User	Admin
Fines Details, filters, export	Yes	Yes	Yes
Executive Summary and charts	No	Yes	Yes
Review status and comment	No	Yes	Yes
Import Dataset	No	No	Yes
User Management and approvals	No	No	Yes

The regular User is stored as `Viewer` in API requests and database roles for compatibility. The interface displays User. A requested role does not grant access before approval. Public signup cannot create an Admin.

Navigation only shows allowed pages. Direct hash links and Back/Forward navigation are checked before rendering. FastAPI independently enforces protected actions; hiding a button is not the security boundary. There is no customer-by-customer record ownership model: authorized readers currently share the fines dataset.

Registration and approval

1. A visitor enters name, email, password, confirmation and requested User/Super User role.
2. FastAPI hashes the password and persists the account as `Pending Approval`.
3. Sign-in is refused while approval is pending.
4. Admin opens User Management, selects the assigned role and approves or rejects.
5. An approved account can sign in with its assigned role. Rejected accounts remain blocked.

Email addresses are normalized to lowercase and must be unique. Account decisions only apply to pending requests; attempting to decide the same request again returns a conflict.

Authentication and sessions

Passwords use Argon2 through pwdlib. Sign-in returns a random bearer token. Only its SHA-256 digest is stored server-side. Sessions expire after eight hours. React stores the token in sessionStorage for that browser tab. Sign-out revokes the server session and clears local state. A 401 response on an authenticated request returns the UI to sign-in.

Fine review

A Super User or Admin opens a record, selects Valid, Invalid or Pending, and supplies a nonblank comment of at most 1,000 characters. FastAPI saves the decision and review history in one transaction. Each review records reviewer identity, old/new status, time and comment. Both expected status and record version are checked; a stale submission returns 409 instead of overwriting a newer decision.

A Pending-to-Pending review is permitted to record missing evidence without resolving the case. User accounts can read the record/history and export, but cannot submit reviews.

3. Frontend structure and UI behavior

Technology and source map

React 19 and Vite provide the application. Recharts renders charts; lucide-react supplies icons. FastAPI handles the middleware and generates Excel exports. No records are generated as a fallback in the browser.

Source	Responsibility
frontend/src/App.jsx	Session state, page guards, navigation and shared actions
frontend/src/pages/LoginPage.jsx	Three role-specific sign-in options
frontend/src/pages/SignUpPage.jsx	Registration and pending-approval feedback
frontend/src/pages/DashboardPage.jsx	Filtered KPIs, charts, source totals and pending drilldown
frontend/src/pages/FinesDetailsPage.jsx	Filters, search, sorting, pagination and export selection
frontend/src/pages/UsersPage.jsx	Admin registration decisions and role assignment
frontend/src/pages/ImportPage.jsx	Validate, review result and explicitly import workbook
frontend/src/services/	HTTP client, service wrappers and permission policy
frontend/src/components/	Layout, charts, dialogs, details drawer and shared UI

Navigation and data behavior

Routes are `#dashboard`, `#fines`, `#import` and `#users`. User lands on Fines Details; Super User and Admin land on Executive Summary. Restricted and unknown routes show an access-restricted screen.

Filters call `/api/fines`. Supported filters are status, customer, operational area, reason, date range and free-text search. Sorting and table pagination are browser-side over the matching API rows. Refresh reloads persisted data. The current implementation retrieves all matches; server-side pagination is a future scaling task.

Executive Summary derives KPIs and charts from the same filtered API rows used by details. Source cards and the pending queue open filtered detail views. `/api/dashboard` is also available as an unfiltered integration endpoint, restricted to Super User/Admin.

All amounts use USD. KPI cards round amounts to whole dollars for display; fine detail and Excel amounts retain cents. Source totals are gross recorded deductions, including all selected statuses; they are not a calculation of confirmed financial liability.

UI states and accessibility

Loading, unavailable and empty results have different feedback. Failed requests offer Retry. Export is disabled while records are unavailable. Operational-area changes reset table pagination. Filter chips and the export confirmation include the operational area.

Review-history failure is shown explicitly rather than described as no history. Modal dialogs and the details drawer support initial focus, Tab containment, Escape and focus restoration. A saving review cannot be dismissed before the request completes. Responsive rules collapse the sidebar and reflow summary cards for narrow screens; wide tables remain horizontally scrollable.

4. Middleware API: identity and access

Base path: `/api`. Development documentation: `http://127.0.0.1:8000/docs`. Authenticated requests send `Authorization: Bearer <token>`. Do not put a database password in React or in a Vite environment variable.

Method and path	Access	Purpose
GET /health	Public	API readiness response
GET /health/database	Admin	Check database connection
POST /auth/signup	Public	Create pending registration
POST /auth/signin	Public	Check credentials, account state and selected role
GET /auth/me	Signed in	Restore current account
POST /auth/logout	Signed in	Revoke bearer session
GET /users/requests	Admin	List pending, approved and rejected accounts
POST /users/{id}/decision	Admin	Approve or reject pending account

Example registration

```
{
  "fullName": "Example User",
  "email": "person@example.com",
  "password": "example-password-123",
  "requestedRole": "Viewer"
}
```

Successful signup returns 201 with `{"status":"Pending Approval"}`. It does not sign the visitor in. The example password is illustrative only.

Sign-in and approval payloads

Sign-in fields are `email`, `password` and `requestedRole`. The role value is `Viewer`, `Super User` or `Admin`. Success returns `token` and a `user` containing `id`, `fullName`, `email` and `role`.

Approval body:

```
{"decision":"approve","assignedRole":"Super User"}
```

Rejection uses `{"decision":"reject"}`. An Admin cannot assign Admin through this public-registration decision endpoint. Create the initial administrator through `backend/bootstrap.py`.

Response semantics

400-series responses represent input or access problems. Common codes: 401 for invalid/expired authentication, 403 for role denial or pending/rejected sign-in, 404 for missing fine, 409 for duplicate email, stale review or already-decided request, 413 for workbook size limits, and 422 for invalid input. FastAPI returns details in the `detail` property. React displays validation messages and handles session expiry.

5. Middleware API: fines and data intake

Method and path	Access	Purpose
GET /fines	All roles	Read filtered records
GET /dashboard	Super User, Admin	Unfiltered counts and source totals
POST /fines	Admin	Create a validated fine
GET /fines/{id}/reviews	All roles	Read audit timeline
POST /fines/{id}/reviews	Super User, Admin	Save decision and comment
POST /imports/validate	Admin	Validate workbook; commit only if requested
POST /exports/fines	All roles	Download server-generated Excel

Filter and record contract

GET `/fines` query parameters: `status`, `customerId`, `sourceArea`, `fromDate`, `toDate`, `reason`, `search`. Dates use YYYY-MM-DD. Reversed date ranges are rejected. Search matches fine ID, customer ID/name and order reference. A successful response is an array of matching records.

Core record properties are `fineId`, `customerId`, `customerName`, `orderReference`, `fineDate`, `fineAmount`, `currencyCode`, `status`, `reasonCategory`, `sourceArea`, `createdAt`, `updatedAt` and `version`. Reviewed records also include `lastReviewedAt`, `lastReviewerName` and `lastReviewComment`. Imported names and order references are blank unless provided through another integration.

Review request

```
{
  "decision": "Invalid",
  "comment": "Delivery evidence does not support this deduction.",
  "expectedStatus": "Pending",
  "expectedVersion": 1
}
```

Success returns the updated fine with incremented version. A conflict requires reloading the current record before another review. The version check protects against two reviewers acting on the same snapshot.

Import and export

Send an `.xlsx` upload as multipart field `file` to `/imports/validate`. Validation alone does not insert anything. Repeat with `?commit=true` only after reviewing validation. An invalid workbook returns `valid:false`, `imported:0` and row-specific errors. No valid subset is imported when any row fails.

An identical previously committed file is rejected by its content digest. This prevents duplicate file uploads, not semantically duplicate rows across edited workbooks. Confirm the business key with the database team before adding row-level deduplication.

Export body: `{"ids":["fine-id"]}` with 1-10,000 IDs. The server rereads those records and writes USD amounts, statuses, reasons, source areas and latest comments. Duplicate requested IDs are removed. Text resembling an Excel formula is written as literal text. The UI exports all matching rows across table pages, rather than only the visible page.

6. Dataset contract and local setup

Workbook rules

The active worksheet must contain these exact headers. Extra columns are ignored. Maximum upload size is 5 MB, expanded workbook size 50 MB and data row count 10,000. Blank rows are skipped. The entire workbook is validated before an import transaction.

Header	Required value
CustomerID	Nonblank customer identifier; text cells preserve leading zeroes
Fines Imposed	Nonnegative USD amount, up to two decimal places
Status	Valid, Invalid or Pending
Reason	Nonblank reason, at most 200 characters
Date	Excel date cell or YYYY-MM-DD
SourceArea	Logistics, Warehouse or Manufacturing

These checks validate structure and types. They do not establish whether a deduction is commercially valid. Automated classification requires separate agreed rules and evidence.

Install and run

Requirements: Python 3.10+, Node 22.12+, npm. Run from the repository root:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r backend/requirement.txt
cd backend
cp .env.example .env
python bootstrap.py
uvicorn main:app --reload --host 127.0.0.1 --port 8000
```

In another terminal:

```
cd frontend
npm ci
npm run dev
```

Open `http://127.0.0.1:5173`. The Vite development proxy sends `/api` requests to port 8000. The ordinary SQLite database starts empty and bootstrap prompts for the first Admin credentials.

Local demonstration mode

`backend/seed_preview.py` explicitly creates sample records and three accounts in a separate `preview.db`; normal startup does not seed these accounts. Start the API from backend/ with `DATABASE_URL=sqlite:///./preview.db ../.venv/bin/uvicorn main:app --host 127.0.0.1 --port 8000`.

Demo credentials: Admin `admin@mars.demo` / `Admin123!`; Super User `reviewer@mars.demo` / `Review123!`; User `viewer@mars.demo` / `Viewer123!`. Keep these known demo accounts out of production. The local preview is currently configured to use this separate database.

7. SQL Server integration handoff

What can stay stable

React keeps calling the same FastAPI contract. SQL Server credentials belong only in backend environment settings. The middleware models and queries are the adapter that must be aligned with the database team's final table names, keys and column types.

Configuration alone is sufficient only if the target database adopts the current middleware schema. If the team supplies a different schema, update ORM models/mapping and queries before connecting; do not assume changing an environment variable maps arbitrary existing tables.

Current logical storage

Table	Current responsibility
users	Account identity, password hash, role, approval state
sessions	Hashed bearer token, account ID and expiry
fines	ID, customer, Decimal(18,2) amount, status, date, area, version and JSON details
reviews	Fine ID and JSON review event with reviewer, comment and statuses
import_batches	Committed file digest to prevent exact-file duplicates

Fine IDs and user IDs are UUID strings. Monetary amounts use a fixed-scale numeric database column. Dates/timestamps are currently ISO strings. Supplementary fine/review fields use SQLAlchemy JSON storage; this is not a finished normalized SQL Server warehouse schema. Confirm whether the final tables will retain JSON or normalize those fields.

Information needed from the database team

- SQL Server host, instance/port, database, schema and approved authentication method.
- Final table/column definitions, primary/foreign keys, nullability and indexes.
- Mapping for CustomerID, Fines Imposed, Status, Reason, Date and SourceArea.
- Unique source/business key for reliable row deduplication and incremental updates.
- Currency, time zone, valid/invalid business rules and source attribution definitions.
- Whether existing fine tables or additional middleware-owned account/audit tables are expected.
- Required permissions, TLS certificate setup and a test database with representative data.

Connection steps

Install Microsoft ODBC Driver 18 on the API host and `pip install -r backend/requirements-sqlserver.txt`. Remove the SQLite `DATABASE_URL` setting. Set `DB_SERVER`, `DB_NAME`, `DB_USERNAME`, `DB_PASSWORD`, and optionally `DB_DRIVER`. Encryption is enabled; certificate trust defaults to verification. The supplied connection path currently uses SQL authentication; other authentication methods need an agreed connection configuration.

Use a test database first. Confirm mapping, run schema changes through the database team's process, test all role/workflow endpoints, validate totals against SQL and then migrate/import approved data. `create_all` creates missing tables; it does not migrate existing schemas or transfer SQLite data. Do not point automatic startup at an unreviewed production schema.

8. Verification and handoff checklist

Automated verification completed

Twelve pytest cases passed against SQLite. Coverage includes signup pending approval, duplicate email, approval/rejection, role denial, wrong-role sign-in, invalid/expired sessions, logout revocation, comment validation, stale review conflicts, search/source/date filters, USD totals, workbook validation, transactional imports, duplicate-file protection, literal-text Excel exports and SQLite persistence after reconnect.

The React production build passed. The production frontend dependency audit reported zero vulnerabilities at this check. Vite reports a large application chunk; code splitting is a future performance improvement, not a build failure.

Run these checks from the repository root:

```
pip install -r backend/requirements-dev.txt
pytest backend/tests -q
cd frontend
npm run build
npm audit --omit=dev
```

Browser verification completed

• All three requested demo accounts signed in with their appropriate role options.

• User-only navigation and direct restricted-page denial were verified.

• Super User could not open import; Admin could open User Management.

• Signup created a pending account and sign-in was blocked before approval.

• Admin approved that account; the approved User then signed in successfully.

• Search returned the expected sample order; the filtered Excel download succeeded.

• A review comment and history persisted, reducing the pending queue and updating summary values during earlier workflow verification.

The browser runs used sample data in the separate preview database. These checks do not certify real-data correctness or SQL Server behavior.

Remaining integration acceptance

1. Receive and review the validated workbook and final SQL table schema.

2. Map fields and confirm keys, currency, dates and source categories.

3. Connect to a non-production SQL Server and repeat the API/workflow tests.

4. Compare counts and deduction totals against independent SQL queries.

5. Verify persistence, duplicate handling and concurrent reviews on SQL Server.

6. Confirm deployment origins, HTTPS, secret handling and backups before release.

Known boundaries

No automatic fine-validity classifier, email/password recovery, customer-specific row ownership, file attachment evidence store, scheduled data sync or production hosting is included. Login throttling, monitoring, schema migrations and load testing need deployment-specific implementation before internet-facing production use. The original repository historically tracked an environment file; current source excludes it, but any real credentials from that history should be rotated.

The frontend and middleware are ready for local demonstration and the defined API contract. Final end-to-end acceptance with real data depends on the SQL Server integration and business validation steps above.